



University of Bisha

College of Computing and Information Technology

GRADUATION PROJECT

Malicious URL Detection with DistilBERT

Supervised by: Dr. Ali Almaliki

Prepared by

Hajar Hajd Alharthi 444802005

Asma Masoud Alshahrani 443801321

Rawan Moshabab Alshahrani 444803633

Shooq Mohammed 443801328

Najla Mohammed Gaashan 444806513

Rawan Mohammed Alharthi 443801335

Anfal Nasser Al Mashout 444804227

Jana Mohsen Alghamdi 443803423

Project Abstract

Malicious URLs are a primary vector for phishing, malware distribution, and website defacement, and traditional detectors that rely on handcrafted lexical rules or blacklists struggle to keep pace with rapidly evolving attacks. This project presents a transformer-based system that classifies URLs into four categories — benign, phishing, malware, and defacement — by fine-tuning DistilBERT directly on raw URL text, removing the need for manual feature engineering. A public Kaggle dataset of roughly 651,000 labeled URLs was split into stratified training, validation, and test sets. The model was fine-tuned end-to-end using the HuggingFace Trainer API and evaluated on a held-out test set of about 97,000 URLs, achieving 98.89% accuracy and a macro F1-score of 0.9835. The results show that a lightweight transformer can reliably detect malicious URLs from text alone, offering a practical alternative to rule-based detection pipelines.

How the Project Was Built

Algorithm Workflow

- 1 Collect and clean the labeled URL dataset
- 2 Tokenize URLs with the WordPiece tokenizer
- 3 Stratified train / validation / test split
- 4 Fine-tune DistilBERT end-to-end
- 5 Validate each epoch, keep the best model
- 6 Evaluate once on the held-out test set

Training ↔ Testing Relationship

Training set (70%)

Used to update DistilBERT's weights via backpropagation.

Validation set (15%)

Used after every epoch to pick the best model — never used to update weights.

Test set (15%)

Held out completely until the very end, for one unbiased final measurement.

Same fixed random seed for every split — so the test score reflects real generalization, not a lucky split.

Malicious URLs Are a Growing Attack Surface

1

Phishing

Fake login pages impersonate trusted brands to steal credentials.

2

Malware

Links trigger drive-by downloads of malicious payloads.

3

Defacement

Compromised sites serve altered or hostile content.

The challenge: classic detectors rely on hand-crafted features (blacklists, WHOIS lookups, lexical rules) that are brittle and slow to update against fast-evolving attacks.

DATASET

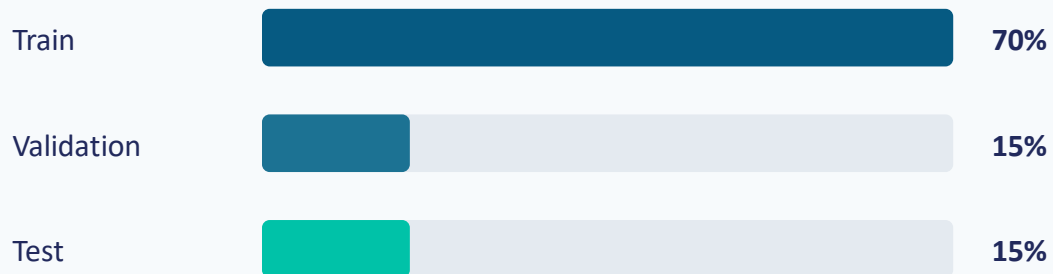
Malicious URLs Dataset (Kaggle)

~651,000

labeled URLs across 4 classes, sourced from the public Kaggle "Malicious URLs Dataset"

Source: Kaggle – Malicious URLs Dataset

Stratified Split (fixed seed 42)



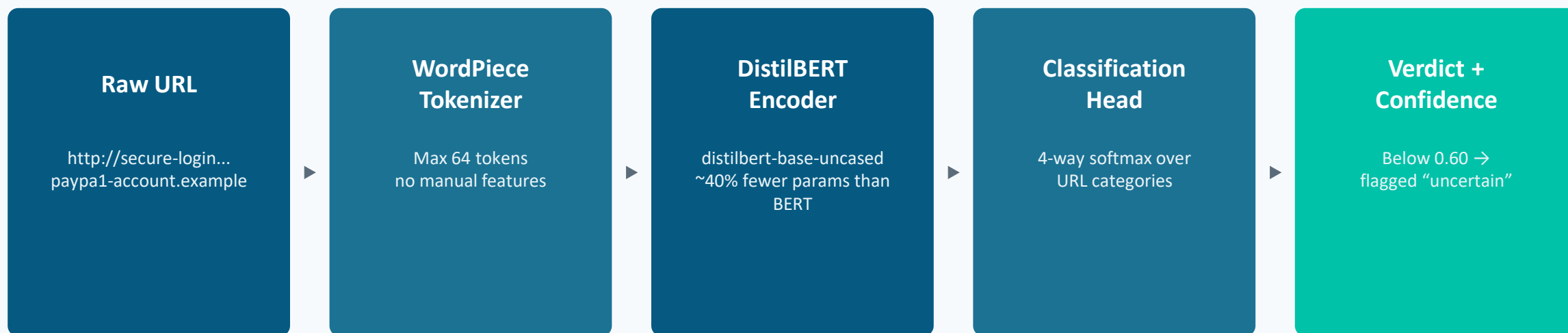
Class proportions preserved in every split

Four Target Classes

- Benign
- Phishing
- Malware
- Defacement

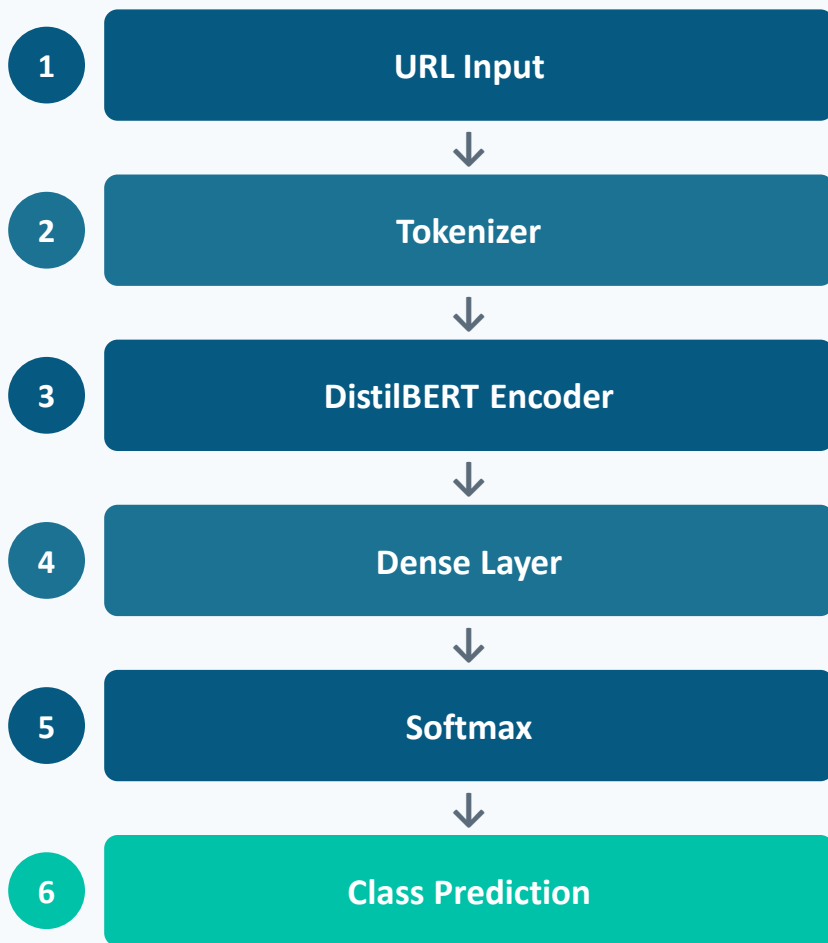
Tokenization: DistilBERT WordPiece tokenizer, max 64 tokens per URL — no manual feature extraction.

End-to-End Classification Pipeline



The same pipeline runs at inference — the CLI, evaluation script, and web demo all call one shared prediction module.

Model Architecture



Layer-by-Layer Specs

URL Input

Raw string passed in as-is — e.g. `http://secure-login.paypa1-account.example/verify`

Tokenizer

DistilBERT WordPiece tokenizer, max 64 tokens, no manual feature extraction

DistilBERT Encoder

6 transformer layers, ~66M parameters — about 40% smaller than BERT-base

Dense Layer

Linear projection from the encoder's hidden size down to 4 output classes

Softmax

Converts the raw logits into a 4-way probability distribution

Class Prediction

Argmax + confidence score; below 0.60 the URL is flagged “uncertain”

Why DistilBERT Over Full BERT?

URLs are short sequences — a lighter encoder trades a small amount of capacity for large gains in speed, without giving up accuracy.

	BERT-base	DistilBERT
Parameters	~110M	~66M (↓ 40%)
Transformer layers	12	6
Suited to short text (URLs)	Adequate	Strong
Inference latency	Higher	Lower
Achieved test accuracy	—	98.89%

Fine-Tuning Configuration

Epochs

3

Learning rate

2e-5

Weight decay

0.01

Optimizer

AdamW

Batch size

16

Framework

HuggingFace Trainer API

Recommended: a CUDA-capable GPU for training (~450K training URLs × 3 epochs) — Google Colab's free T4 works well. Inference runs comfortably on CPU.

RESULTS

Test-Set Performance

98.89%

overall accuracy across ~97,000 held-out URLs

98.61%

Macro Precision

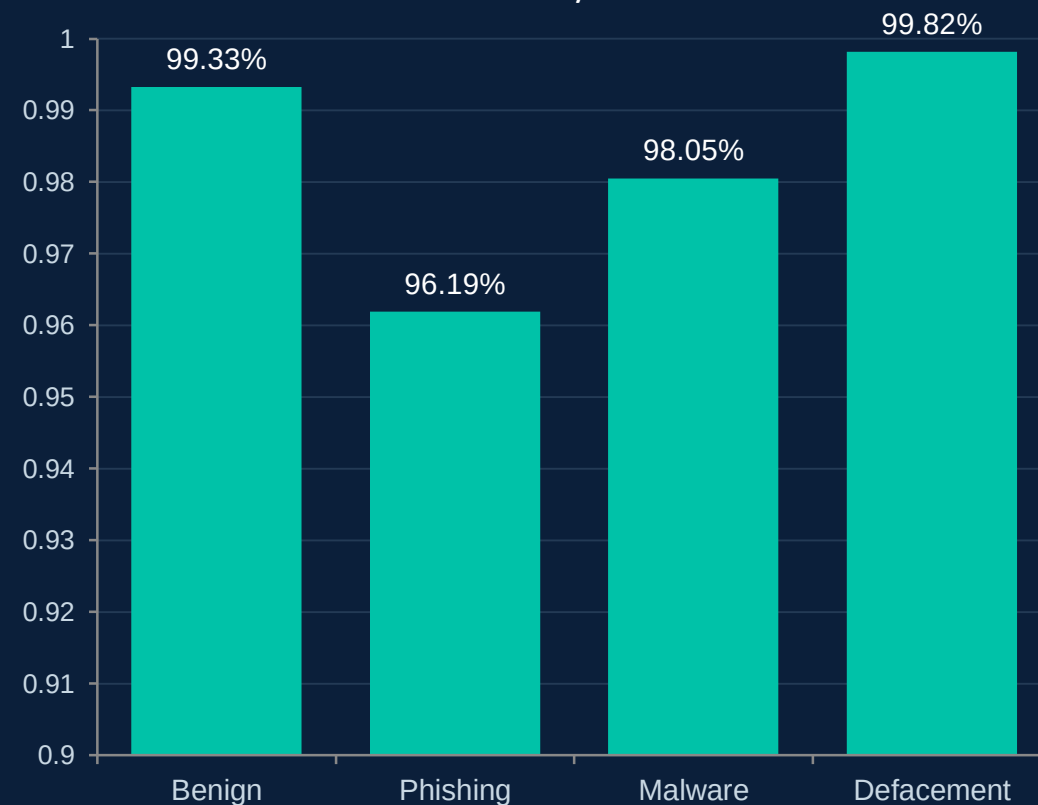
98.09%

Macro Recall

98.35%

Macro F1-Score

F1-Score by Class



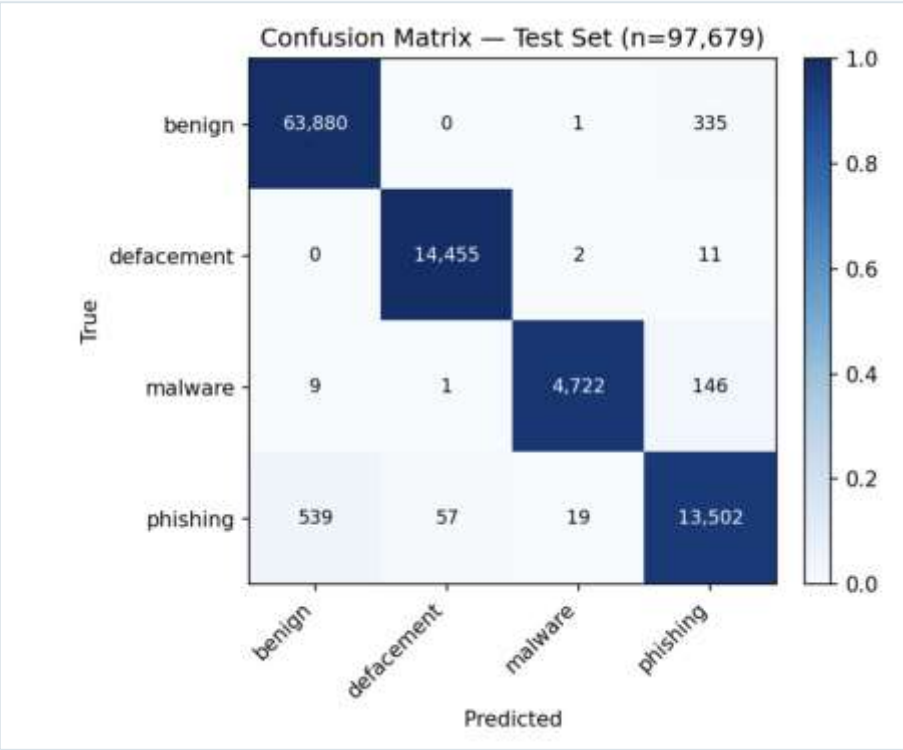
Per-Class Metrics

Class	Precision	Recall	F1-Score
Benign	0.9922	0.9944	0.9933
Phishing	0.9641	0.9596	0.9619
Malware	0.9908	0.9705	0.9805
Defacement	0.9974	0.9991	0.9982
Macro Avg	0.9861	0.9809	0.9835

Phishing is the hardest class (borderline URLs resemble benign login pages) — still 96%+ F1. Defacement is near-perfect due to distinctive lexical patterns.

RESULTS

Confusion Matrix



Most predictions lie on the main diagonal, indicating excellent classification performance.

Working Demo — Live Classification

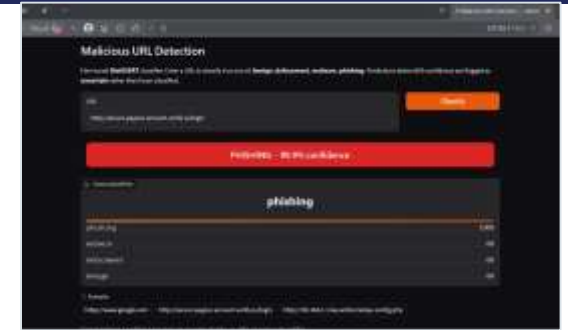
Real-Time URL Classification

- Enter any URL and get a real-time prediction
- Confidence score with full class-probability bars
- Predictions below the 0.60 threshold flagged as “uncertain” rather than force-classified
- Runs the exact same trained model used for the reported test results

Live Demo — Actual Screenshots



Enter a URL to classify



Phishing detected — 99.9% confidence

Captured directly from the deployed Gradio web app — not a mockup

Key Takeaways

98.89% accuracy

Classifying URLs into 4 categories using raw text alone, with zero handcrafted features.

DistilBERT is enough

A lighter, faster transformer matches the job — URLs are short sequences that don't need full BERT.

Reliable by design

Stratified splits and a held-out test set mean the reported score reflects true generalization.

Working end-to-end system

A trained model, an evaluation pipeline, and a live web demo — not just a notebook.
Ready to classify unseen URLs in real time.

Where This Project Can Go Next

1

Deploy as a browser extension

Bring real-time URL checks directly into the user's browsing experience.

2

Train on continuously updated URL datasets

Keep the model current against newly emerging phishing and malware patterns.

3

Compare with larger transformer models

Benchmark against BERT-large or RoBERTa to measure the accuracy/speed trade-off.

Thank You

Questions & Discussion

Hajar Hajd Alharthi · Najla Mohammed Gaashan · Asma Masoud Alshahrani · Rawan Mohammed Alharthi

Rawan Moshabab Alshahrani · Anfal Nasser Al Mashout · Shooq Mohammed · Jana Mohsen Alghamdi

Supervised by Dr. Ali Almaliki